



<https://blog.yanlincs.com/ai-system/>

Self-hosted,
accessible website

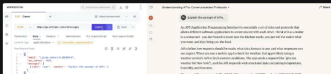
AI Systems & Infrastructure

Basics of streamlined interaction with AI systems, peripherals that enable such interaction, and deployment of AI systems on real-world infrastructure.

This is also the companion literature for the DAK13 - AI Systems & Infrastructure course at Aalborg University in the form of blog posts.

	Introduction to AI Systems & Infrastructure
Phase A	Interact with AI Systems
Module A.1	API Fundamentals
Module A.2	Advanced APIs in the Era of AI
Module A.3	Wrap AI Models with APIs
Phase B	Infrastructure & Deployment of AI
Module B.4	AI Compute Hardware
Module B.5	Packaging & Containerization
Module B.6	Cloud Deployment
Module B.7	Edge & Self-hosted Deployment

Example: When you send a request to OpenAI/Anthropic's API, you wait for a few seconds for the complete response to appear. However, when you interact with ChatGPT/Claude on their official web/mobile app, their responses are continuously streamed to you word-by-word. In reality, the streaming behavior is also achievable through APIs.



Everything in one place

Relatable, fun analogy



The benefits of containers quickly made containerization become the industry standard for large-scale software deployment. Today, there is a very high chance that one of the applications you use everyday is running in containers. It is reported that by 2025, container usage in the IT industry has reached 92%. With the help of containers, companies can deploy updates without downtime, handle more users by scaling automatically, and run the same software reliably across different hardware infrastructures.

Code examples with explanation

Multi-media (images & videos)

Unlike SSE which is built on top of HTTP, WebSocket is a communication protocol of its own. For two applications to establish a WebSocket connection, one application first sends a standard HTTP request with upgrade headers, while the other application agrees to upgrade and maintains the connection through the WebSocket lifecycle. To create a WebSocket connection in Python, we no longer can use the `requests` package since it is specifically built for HTTP. Instead, we have to use `websocket` package. Below is a basic example of connect to OpenAI's real-time API:

```
import os
import json
import websocket

OPENAI_API_KEY = os.getenv("API_KEY")
url = "wss://api.openai.com/v1/realtime?model=gpt-4o-realtime-preview-2024-12-17"

headers = [
    "Authorization: Bearer " + OPENAI_API_KEY,
    "OpenAI-Beta: realtime=v1"
]

def on_open(ws):
    print("Connected to server.")
    # Send a request to the API when the connection opens
    payload = {
        "type": "response.create",
        "response": {
            "modalities": ["text"],
            "instructions": "Say hello!"
        }
    }
    ws.send(json.dumps(payload))

def on_message(ws, message):
    data = json.loads(message)
    print("Received event:", json.dumps(data, indent=2))

ws = websocket.WebSocketApp(
    url,
    header=headers,
    on_open=on_open,
```